BTK AKADEMİ

Her an,
her yerde
öğren !

Yapay Zeka ve Üretken Modeller Atölyesi - 3

Dr.Öğr.Üyesi Furkan Göz

Derin Öğrenmenin Yükselişi (Google Trends)

- 2012'deki AlexNet başarısı sonrası artan **GPU işlem gücü** ve büyük veri setlerinin (**Big Data**) erişilebilirliği Yapay Sinir Ağlarının (Derin Öğrenme) çağını başlatmıştır.

AlexNet ve ImageNet

ImageNet Nedir?

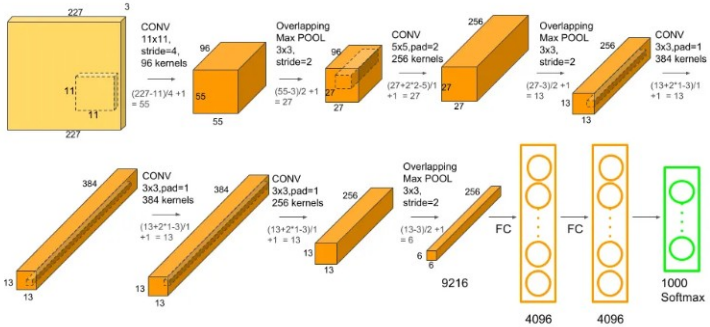
- 14 Milyon 1000 sınıflı görüntü içeren büyük bir veri setidir.
- Görüntü sınıflandırma algoritmalarını karşılaştırmak için kullanılır.

AlexNet Nedir?

- 2012 yılında **Alex Krizhevsky, Ilya Sutskever ve Geoffrey Hinton** tarafından geliştirilen derin evrişimsel sinir ağıdır (CNN).
- GPU kullanarak eğitilen ilk büyük ölçekli derin öğrenme modellerinden biridir.
- 2012 ImageNet yarışmasında hata oranını yaklaşık **%26'dan %15'e** düşürerek birinci olmuştur.

ImageNet

AlexNet


227
227
3
11
11
CONV
11x11,
stride=4,
96 kernels
(227-11)/4 +1
= 55
96
55
55
Overlapping
Max POOL
3x3,
stride=2
(55-3)/2 +1
= 27
96
27
27
CONV
5x5,pad=2
256 kernels
(27+2*2-5)/1
+1 = 27
256
27
27
Overlapping
Max POOL
3x3,
stride=2
(27-3)/2 +1
= 13
256
13
13
CONV
3x3,pad=1
384 kernels
(13+2*1-3)/1
+1 = 13
384
13
13
CONV
3x3,pad=1
384 kernels
(13+2*1-3)/1
+1 = 13
384
13
13
CONV
3x3,pad=1
256 kernels
(13+2*1-3)/1
+1 = 13
256
13
13
Overlapping
Max POOL
3x3,
stride=2
(13-3)/2 +1
= 6
256
6
6
9216
FC
4096
FC
4096
1000
Softmax

Yapay Sinir Ağı (Artificial Neural Network - ANN)

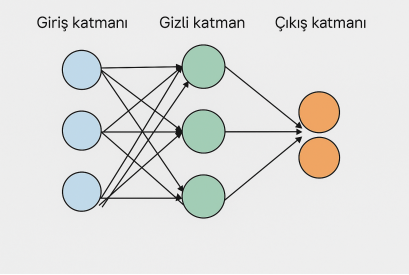
- Biyolojik sinir sisteminden esinlenilerek geliştirilmiş, verilerdeki karmaşık örüntüleri kendi kendine öğrenebilen matematiksel modellerdir.
- **Öğrenme Akışı:** Girdi $\rightarrow$ Ağırlıklandırılmış Toplam $\rightarrow$ Aktivasyon $\rightarrow$ Çıktı.
- Çok sayıda nöron (düğüm) ve katmandan (Giriş, Gizli, Çıkış) oluşur.

Neden Güçlüdür?

- Teorik olarak yeterli sayıda gizli nörona ve doğrusal olmayan aktivasyona sahip bir YSA herhangi bir matematiksel fonksiyonu (ne kadar karmaşık olursa olsun) öğrenebilir ve taklit edebilir.

Katman Türleri

- **Giriş Katmanı (Input Layer):** Ham verinin (sayısal vektörler) ağa girdiği kapıdır. Hesaplama yapılmaz.
- **Gizli Katmanlar (Hidden Layers):** Ağın öğrenme ve özellik çıkarma (feature extraction) görevini yaptığı bölümdür. Ağırlıklı toplama ve aktivasyon burada gerçekleşir.
- **Çıkış Katmanı (Output Layer):** Problemin türüne göre (sınıflandırma veya regresyon) nihai kararın tahmin edildiği katmandır.



Yapay Nöron Nasıl Çalışır?

- Nöron kendisine gelen verileri (girdiler) **ağırlıklarla (weights) çarpar**. Ağırlıklar bilginin ne kadar önemli olduğunu belirler.
- Çıkan değerlerin toplamına bir **sapma (bias)** değeri eklenir. Bu doğrusal (lineer) bir işlemdir.
- Sonuç bir **aktivasyon fonksiyonu** ile doğrusal olmayan bir çıktı elde edilir ve bir sonraki katmana iletilir.

Matematiksel Formül

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \quad \Rightarrow \quad a = f(z)$$

- x_i: Girdiler (Features), w_i: Ağırlıklar (Öğrenilen önem dereceleri)
- b: Bias (Sapma), $f(z)$: Aktivasyon fonksiyonu

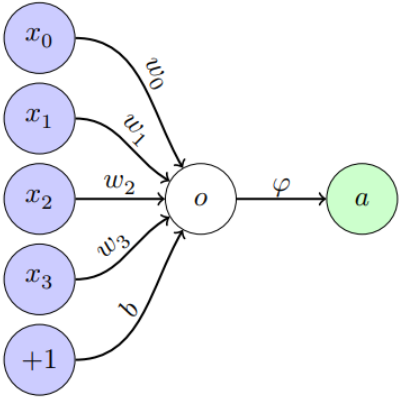
Bias (Sapma) Neden Eklenir?

- Modelin yalnızca girişlere bağlı kalmadan aktivasyon eşiğini sağa veya sola kaydırarak *daha esnek* bir karar sınırı öğrenmesini sağlar.
- Bias olmasaydı ($b = 0$) modelin çizeceği tüm karar doğruları orijinden (0,0) geçmek zorunda kalırdı.

Örnek: $z = w_1 x_1 + w_2 x_2 + b$

- Eğer $w_1 = 1$, $x_1 = 2$ ve $b = 0$ ise $\rightarrow z = 2$
- Aynı durumda $b = +3$ olursa $\rightarrow z = 5$

En Temel Yapı: Perceptron

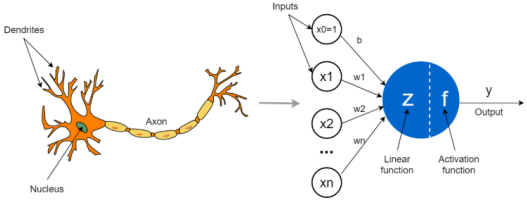


- **Perceptron:** 1957'de Frank Rosenblatt tarafından önerilen yapay sinir ağlarının en küçük ve en temel yapı taşıdır (Tek nöronlu ağ mimarisi).

Biyolojik Nörondan Yapay Nörona

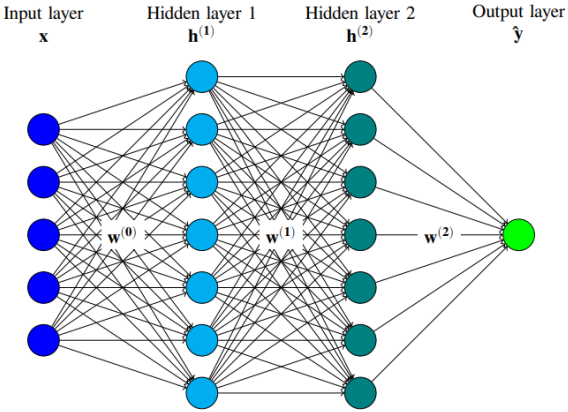
Biyolojik / Yapay Eşleşmesi

- **Dendritler** $\rightarrow$ Girdiler (x_n)
- **Sinapslar** $\rightarrow$ Ağırlıklar (w_n)
- **Hücre Gövdesi** $\rightarrow$ Lineer Toplam (z)
- **Akson** $\rightarrow$ Aktivasyon Çıktısı ($f(z)$)



Yapay nöron veriyi işleyip aktivasyon fonksiyonundan geçirdikten sonra sonucu bir sonraki katmandaki nöronlara aktarır.

Çok Katmanlı Algılayıcı (Multilayer Perceptron - MLP)



- Tek bir nöron (Perceptron) sadece doğrusal problemleri çözebilir. Doğrusal olmayan karmaşık problemleri çözmek için **Gizli Katmanlara (Hidden Layers)** sahip MLP mimarisi gereklidir.

Girdi ve Çıktı Katmanlarında Boyutlandırma

1. Girdi Katmanı (Input Layer)

- Nöron sayısı veri setindeki **bağımsız (açıklayıcı) değişken (feature) sayısına** eşittir.
- *Örnek:* Veri setinde Yaş, Kilo, Boy ve Tansiyon özellikleri varsa → Ağa veri giriş kapısı 4 nöronlu olmalıdır.

2. Çıktı Katmanı (Output Layer)

- Nöron sayısı hedef göreve bağlıdır.
- **Regresyon:** 1 nöron (Tahmin edilen sürekli rakamsal değeri üretir).
- **İkili Sınıflandırma (Binary):** 1 nöron (Sigmoid).
- **Çok Sınıflı Sınıflandırma (Multiclass):** Sınıf sayısı kadar nöron (Softmax).

Örnek Problem: 3 Sınıflı Sınıflandırma (Çok Sınıflı)

- Veri setinde 4 adet bağımsız (açıklayıcı) özellik (feature) bulunmaktadır.
- Hedef değişken (target) 3 farklı sınıftan birine aittir $\rightarrow$ Çok sınıflı (Multi-class) sınıflandırma problemi.

Ağ Mimarisi

- **Giriş katmanı:** 4 nöron (4 özellik için 4 kapı).
- **Çıktı katmanı:** 3 nöron (Her bir sınıf için 1 çıkış kapısı).
- **Çıkış Aktivasyonu:** `Softmax`

Amaç: Ağa giren her bir veri örneği için verinin 3 sınıfa ait olma olasılıklarını matematiksel olarak hesaplamak.

Gizli Katman (Hidden Layer) Sayısı ve Nöronlar

- Girdi ve çıktı katmanındaki nöron sayısı veri setine bağlı olarak belirlenir.
- **Gizli katman sayısı ve her katmandaki nöron sayısı** modelin kapasitesini (Capacity) belirleyen hiperparametrelerdir.

Nasıl belirlenir?

- Deneme yanılma (Grid Search, Random Search, Keras Tuner).
- Kabul görmüş standart mimariler.
- Otomatik mimari arama algoritmaları (AutoML / NAS).

Trade-off

Fazla nöron veya derin katmanlar → Yüksek ezberleme (Overfitting) riski ve yavaş eğitim.

Az nöron veya sığ ağlar → Yetersiz öğrenme (Underfitting) ve düşük kapasite.

Veri Temsili: Iris Veri Seti

```python
from sklearn.datasets import load_iris
import numpy as np
import pandas as pd

data = load_iris()
X = data.data
y = data.target

input_dim = X.shape[1] # Sutun sayisi (Ozellikler)
output_dim = len(np.unique(y))

print(f"Giris Noronu Sayisi: {input_dim}")    # Cikti: 4
print(f"Cikis Noronu Sayisi: {output_dim}")   # Cikti: 3
```

Neden 4 Giriş, 3 Çıkış Nöronu?

Giriş Katmanı (Input)

- Sepal uzunluğu, sepal genişliği, petal uzunluğu, petal genişliği.
- Toplam 4 özellik var → Bu yüzden veri ağa 4 koldan (4 nöron) girmelidir.

Çıkış Katmanı (Output)

- 3 çiçek sınıfı: Setosa, Versicolor, Virginica.
- Model her bir çiçeğe ne kadar benzediğini ayrı ayrı söyleyeceği için 3 çıkış nöronu gerekir.
- Aktivasyon: **Softmax** (Örn: Setosa %80, Versicolor %15, Virginica %5).

Aktivasyon Fonksiyonu Nedir?

- Yapay sinir ağında her bir nöronun matematiksel toplamı hesapladıktan sonra **çıktı üretip üretmeyeceğine (tetiklenip tetiklenmeyeceğine)** karar veren filtredir.

Ağın öğrenme kapasitesini sadece düz çizgilerle sınırlı kalmaktan kurtarıp **doğrusal olmayan (non-linear)** karmaşık problemleri çözebilmesini sağlamak amaçlanır.

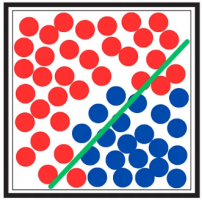
Biyolojik İlham: Aksiyon Potansiyeli

- Aktivasyon fonksiyonlarının temel fikri biyolojik nöronlardaki **Aksiyon Potansiyeli** kavramına dayanır.
- **Aksiyon potansiyeli:** Bir sinir hücresinde (nöron) biriken elektriksel sinyalin belirli bir eşik değerini (threshold) aştığı anda patlayarak diğer nöronlara aktarılmasıdır.
- Yapay sinir ağlarında bu mekanizma nörona gelen ağırlıklı toplamın (z) bir fonksiyon ($f(z)$) aracılığıyla filtrelenerek diğer katmana iletilmesiyle modellenir.

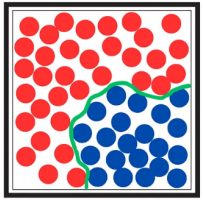
Doğrusal Olmayanlık

- Gerçek dünya verisi (ses dalgaları, pikseller, kelime vektörleri) doğrusal bir çizgiyle ayrılamayacak kadar iç içe geçmiş **doğrusal olmayan (non-linear)** yapıdadır.
- Sadece doğrusal (linear) regresyonla bir resimdeki kedi ve köpeği birbirinden ayıralamaz.
- **Örnek:** Köpek/kedi tanıma problemi – Aynı türdeki hayvanlar farklı poz, açı, renk ve arka planda olabilir. Aktivasyon fonksiyonları sayesinde model uzayı eğip bükerek bu sınıfları ayıracak sınırları (decision boundaries) çizmeyi öğrenir.

Aktivasyon



Neural Network without an Activation Function



Neural Network with an Activation Function

- **Aktivasyonsuz:** Ne kadar eğitilirse eğitilsin sadece düz bir çizgi çizebilir.
- **Aktivasyonlu:** Düzlemi bükerek karmaşık veri kümesini ayırır.

Öğrenme Süreci ve Gradyan (Türev) Yayılımı

- Sinir ağı eğitilirken model hata yapar. Bu hatayı düzeltmek için **Geri Yayılım (Backpropagation)** algoritması ile ağın en sonundan en başına doğru ağırlıklar güncellenir.
- Güncelleme **Aktivasyon Fonksiyonunun Türevi (Gradyanı)** alınarak yapılır.
- **Kaybolan Gradyan Problemi:** Eğer aktivasyon fonksiyonunun türevi çok küçük çıkarsa hata sinyali geriye giderken sıfıra yaklaşır ve ağın başındaki nöronlar öğrenmeyi durdurur.

Karar Verme Mekanizması ve Filtreleme

- Her nöron önceki katmandan gelen Ağırlıklı Toplamı hesaplar.
- Aktivasyon fonksiyonu bu toplama bakarak nöronun **ne kadar şiddetle çıkış üreteceğine** karar verir.
- Böylece verideki bazı özellikler (features) vurgulanıp ön plana çıkarılırken gereksiz gürültüler (noise) bastırılıp sıfırlanır.
- **Örnek:** Görüntü işlemede bir nöron kenarı diğer nöron kırmızıyı algılayabilir. Aktivasyon fonksiyonu bu algının şiddetini ayarlar.

Hiyerarşik Özellik Çıkarımı (Hierarchical Feature Extraction)

- Derin sinir ağları katman katman öğrenir. Aktivasyon fonksiyonları bu katmanlar arası bilginin dönüşerek üst seviyeye çıkmasını sağlar.
- İlk katmanlar verideki çok basit örüntüleri yakalar, üst katmanlar ise bu basit örüntüleri birleştirerek karmaşık nesneleri tanır.

Örnek: Yüz Tanıma Sistemi

- **Alt Katmanlar:** Sadece piksellerdeki düz ve eğri çizgileri (kenarları) öğrenir.
- **Orta Katmanlar:** Çizgileri birleştirip göz, kulak, burun şekillerini öğrenir.
- **En Üst Katmanlar:** Tüm organları birleştirip İnsan Yüzünü öğrenir.

İleri Yayılım (Forward Propagation)

Girdiler ve öğrenilmiş ağırlıklar

- Yaş (x_1) = 2, Kilo (x_2) = 3
- Ağırlık (w_1) = 0.5, Ağırlık (w_2) = 0.3
- Bias (b) = 2

1. Adım: Ağırlıklı Lineer Toplam (z)

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b = (0.5 \cdot 2) + (0.3 \cdot 3) + 2 = \mathbf{3.9}$$

2. Adım: Sigmoid Aktivasyon ($f(z)$)

$$f(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-3.9}} \approx \mathbf{0.98}$$

Aktivasyon Çıktısının Yorumu

- Sigmoid fonksiyonu **0 ile 1 aralığında** değer üretir.
- 0.98 değeri istatistiksel bir Olasılık olarak yorumlanır.
- Model diyor ki: Gelen verinin **%98 ihtimalle** hedef sınıfa ait olduğundan eminim.

Aktivasyon Fonksiyonu Olmasaydı?

- Eğitilen ağın katmanları arasına hiçbir aktivasyon fonksiyonu olmazsa:
  - Matematiksel olarak $f(f(x)) = W \cdot (W \cdot x + b) + b$ zinciri oluşur.
  - Lineer cebir kuralları gereği iki lineer matrisin çarpımı bir lineer matrise eşittir.
  - **100 katmanlı bir ağ tek katmanlı basit bir lineer regresyon modeline dönüşür.**
- Derinliğin (Deep Learning'in Deep kelimesinin) hiçbir anlamı kalmaz.
- **Sonuç:** Model sadece düzlem veya doğru çizebilir. Karmaşık görevleri (çeviri yapma, resim üretme) yapamaz.

Softmax Aktivasyon Fonksiyonu ve Logits

Amaç: Çok sınıflı sınıflandırma problemlerinin **çıkış katmanında** yer alır. Ağdan çıkan ham tahmin skorlarını (Logits) olasılığa dönüştürür.

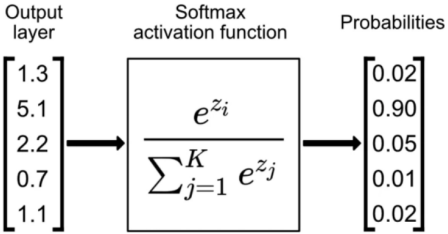
Nasıl?

- Modelin ürettiği sayılara **Logits** denir.
- Softmax logitleri alır, üstel fonksiyonla (e^z) pozitif hale getirir.
- Daha sonra hepsini toplayıp birbirine bölerek **toplamları her zaman %100 (1.0) eden olasılık dağılımı** oluşturur.

Formül

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad (K\text{: Toplam sınıf sayısı})$$

Softmax


Output layer
Softmax activation function
Probabilities
1.3
5.1
2.2
0.7
1.1
$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
0.02
0.90
0.05
0.01
0.02

Aktivasyon Fonksiyonları

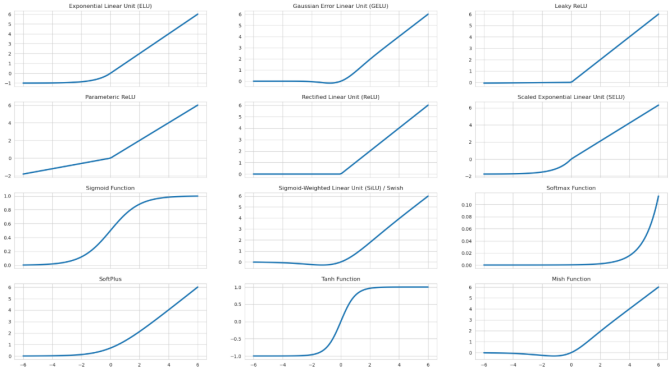
- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$
  - Çıktıyı $(0, 1)$ aralığına getirir. İkili (Binary) sınıflandırmanın son katmanında kullanılır.
- **Tanh:** $f(x) = \tanh(x)$
  - Çıktıyı $(-1, 1)$ aralığına getirir.
- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$
  - Gelen sayı 0'dan küçükse 0 yapar, büyükse aynen geçirir. Matematiksel olarak basittir ama doğrusal olmayanlık sağlar.

Hangi Durumda Hangi Fonksiyon Seçilmeli?

- **Sigmoid:** Çıkış katmanında olasılık isteniyorsa kullanılır. Ara katmanlarda *Kaybolan Gradyan*'a sebep olabilir.
- **ReLU:** Ara (Gizli) katmanların varsayılan (default) olarak kullanılır.
- **Leaky ReLU / ELU:** ReLU'nun dezavantajı eksi değerleri tamamen yok eder. Alternatif olarak eksi değerler için kullanılır.
- **GELU (Gaussian Error Linear Unit):** Transformer mimarilerinde kullanılır.

Doğru aktivasyon seçilmezse yapay sinir ağı öğrenemez.

Aktivasyon Fonksiyonları



Veri Ön İşleme (Data Preprocessing) Nedir?

- Makine öğrenmesi modelleri ham veriyi doğrudan işleyemez.
- Bu yüzden veriler modele uygun şekilde temizlenip dönüştürülmelidir.
- Bu adıma **veri ön işleme (preprocessing)** denir.

Veri Ön İşlemede Yapılan Temel Adımlar

- **Eksik Veri Temizleme:** Boş (NaN) değerlerin doldurulması veya silinmesi.
- **Etiketleme:** Kategorik verileri sayısallaştırma (Label Encoding / One-Hot Encoding).
- **Özellik Ölçekleme:** Tüm sayısal verileri ortak ölçeğe getirme (Örnek: 0 ile 1 aralığı).
- **Normalizasyon / Standartlaştırma:** Verinin varyansını ve ortalamasını düzenleme.
- **Veriyi Bölme:** Veri setini eğitim ve test olarak ayırma.

Ön işleme yapılmadan model eğitmek genellikle başarısızlıkla sonuçlanır.

Keras İçin Veri Ön İşleme Örneği

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

X, y = load_iris(return_X_y=True)
y = to_categorical(y) # Etiketleme islemi

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # Olcekleme islemi
X_test = scaler.transform(X_test)
```

TensorFlow ve Keras ile Sinir Ağı Modeli Tanımı

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, InputLayer

model = Sequential([
    InputLayer(shape=(10, )),
    Dense(units=20, activation="relu"),
    Dense(units=3, activation="softmax")
])

model.summary()
```

- **Keras API:** TensorFlow arka planında (C++ ve CUDA) çalışan karmaşık matematiksel grafikleri basitleştiren bir arayüzdür.
- **Sequential (Sıralı) Mimari:** Verinin katmanlar arasında baştan sona doğru doğrusal bir çizgide ilerlediği en yaygın ağ modelidir.

Model Mimarisi ve Parametre Hesaplama

- **Giriş Katmanı**
  - `InputLayer(shape=(10, ))` → Modelin her defasında 10 özellikten oluşan tek boyutlu bir vektör beklediğini belirtir.
- **Gizli Katman (Tam Bağlı - Fully Connected)**
  - `Dense(20, activation="relu")` → 20 nöronluk bir yapıdır. Önceki katmandaki tüm nöronlara bağlı olduğu için **Dense (Yoğun)** adını alır.
- **Çıkış Katmanı**
  - `Dense(3, activation="softmax")` → 3 farklı sınıf için olasılık dağılımı (%100'e tamamlanan) üretir.
- **model.summary() Fonksiyonu**
  - Ağın öğrenilecek **Toplam Parametre (Ağırlık + Bias)** sayısını gösteren hata ayıklama aracıdır.

Uygulama: Iris Veri Seti İçin MLP Modeli

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, InputLayer

model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(units=100, activation="relu"),
    Dense(units=3, activation="softmax")
])
```

- **4 giriş özelliği:** Çiçeklerin fiziksel ölçüleri ağa bu kapıdan girer.
- **100 nöronlu ReLU katmanı:** Çiçek yaprakları arasındaki karmaşık doğrusal olmayan geometrik ilişkileri öğrenir.
- **3 sınıflı Softmax çıktısı:** Verilen çiçeğin hangi türe ait olduğunu bir olasılık vektörü olarak dışarı aktarır.

Kayıp Fonksiyonu (Loss / Cost Function) Nedir?

- Modelin yaptığı tahmin ile gerçek (olması gereken) değer arasındaki matematiksel farkı ölçen fonksiyondur.
- **Maliyet Fonksiyonu (Cost Function)** veya **Amaç Fonksiyonu (Objective Function)** olarak bilinir.
- **Amaç:** Eğitim süreci boyunca bu kayıp değerini mümkün olan en düşük seviyeye (minimum noktaya) indirmektir.
- Kayıp sıfıra ne kadar yakınsa modelin ağırlıkları gerçeğe o kadar uygun optimize edilmiş demektir.

Kayıp Fonksiyonu

- Sinir ağı bir tahmin vektörü üretir. Örnek tahmin: `[0.2, 0.3, 0.5]`
- Veri setindeki gerçek sınıf etiketi: `[0, 0, 1]`
- Kayıp fonksiyonu bu iki vektör arasındaki farkı skaler (tek bir sayısal) değere dönüştürür.
- Bu skaler fark ağın ağırlıklarının hangi yönde ve ne büyüklükte güncelleneceğini belirleyen **türev (gradyan)** hesaplamasında temel referans noktasıdır.

Kayıp Fonksiyonları

1. Mean Squared Error (MSE - Ortalama Kare Hatası)

- **Regresyon** (sürekli değer tahmini) problemlerinde kullanılır.
- Hataların karesi alındığı için büyük sapmaları daha çok cezalandırır.

2. Binary Crossentropy (İkili Çapraz Entropi)

- **İkili sınıflandırma** (Hasta/Sağlam, Spam/Normal) için uygundur.
- Çıkış katmanındaki **Sigmoid** aktivasyonu ile uyumludur.

3. Categorical Crossentropy (Kategorik Çapraz Entropi)

- **Çok sınıflı sınıflandırmalarda** kullanılır.
- Çıkış katmanındaki **Softmax** aktivasyonu ile birlikte çalışarak olasılık dağılımları arasındaki farkı hesaplar.

Çok Sınıflı Kayıp Seçimi

- Çok sınıflı veri setlerinde çıkış katmanı `Dense(3, activation="softmax")` şeklindedir.
- Kullanılacak kayıp fonksiyonu veri ön işleme adımında etiketlerin (y) nasıl kodlandığına göre değişiklik gösterir:
- **One-Hot Encoding kullanıldıysa** (Örn: `[0, 1, 0]`):
  - Kayıp fonksiyonu `categorical_crossentropy` olmalıdır.
- **Label Encoding / Integer kullanıldıysa** (Örn: `1`):
  - Kayıp fonksiyonu `sparse_categorical_crossentropy` olmalıdır.

Regularization (Düzenlileştirme) Nedir?

- Sinir ağları güçlü modellerdir ve kolayca ezberleyebilirler (overfitting).
- Regularization **genelleme** kabiliyetini artırmasını sağlar.
- **Amaç:** Modelin karmaşıklığını kontrol altında tutarak yeni verilerdeki performansını yükseltmektir.

Regularization Yöntemleri

1. L1 ve L2 Regularization

- Ağırlıklara matematiksel ceza (penalty) uygulanır.
- **L1 (Lasso):** Ağırlıkların toplam mutlak değeri baz alınır.
- **L2 (Ridge):** Ağırlıkların karelerinin toplamı baz alınır.

2. Dropout

- Eğitim sırasında rastgele seçilen bazı nöronlar devre dışı bırakılır.
- Ağa yapısal çeşitlilik kazandırır ve nöronların birbirine aşırı bağımlı olmasını (ezberlemeyi) engeller.

Keras ile Dropout Kullanımı

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, InputLayer

model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(64, activation="relu"),
    Dropout(0.5), # Egitim sirasinda %50 noron gecici olarak
        kapatilir
    Dense(3, activation="softmax")
])
```

- `Dropout(0.5)` kullanımı aşırı öğrenme (overfitting) riskini önemli ölçüde azaltır.

Keras ile L2 Regularization Kullanımı

```
from tensorflow.keras.regularizers import l2
from tensorflow.keras.layers import Dense

# kernel_regularizer parametresi ile agirliklara ceza uygulanir
Dense(64, activation="relu", kernel_regularizer=l2(0.01))
```

- `l2(0.01)` parametresi ile her ağırlık için belirlenen oranda ceza eklenir.
- Model çok büyük ağırlıklar öğrenemez ve aşırı karmaşıklık önlenmiş olur.

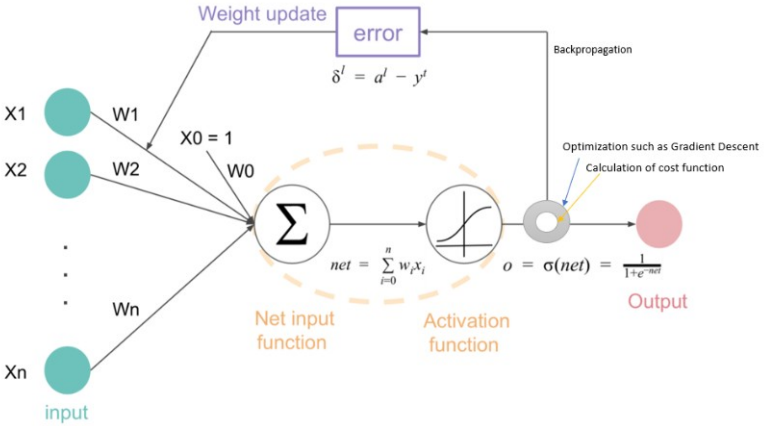
Backpropagation (Geri Yayılım) Algoritması

- Sinir ağlarının öğrenmesini sağlayan algoritmadır.
- Sinir ağı çıktısıyla gerçek değer karşılaştırılır, hata hesaplanır.
- Hata ağın çıkışından girişine doğru geriye doğru yayılır.
- Katmanlardaki ağırlıklar bu hatanın sorumluluk oranına göre güncellenir.
- **Amaç:** Ağırlıkları öyle ayarla ki kayıp fonksiyonu (hata) minimum olsun.

Geri Yayılım

- **1. İleri Yayılım (Forward Pass):** Girdi verisi ağdan geçer ve bir çıkış (tahmin) üretilir.
- **2. Kayıp Hesabı (Loss Calculation):** Üretilen çıkış ile gerçek değer arasındaki fark matematiksel olarak ölçülür.
- **3. Geriye Yayılım (Backward Pass):** Hata türevler kullanılarak ağırlıklara doğru geriye yayılır.
- **4. Gradient Descent:** Ağırlıklar hatayı azaltacak yönde güncellenir.

Geri Yayılım



Gradient Descent Örneği

Parametreler: $x = 2$, $w = 0.3$ **Gerçek:** $y = 1$ **Öğrenme Oranı:** $\eta = 0.1$

1. Model Çıktısı

$$\begin{aligned}\hat{y} &= wx \\ &= 0.3 \times 2 = 0.6\end{aligned}$$

2. Hata (MSE)

$$\begin{aligned}L &= \frac{1}{2}(y - \hat{y})^2 \\ &= \frac{1}{2}(1 - 0.6)^2 = 0.08\end{aligned}$$

3. Gradyan

$$\begin{aligned}\frac{\partial L}{\partial w} &= (\hat{y} - y)x \\ &= (0.6 - 1) \times 2 = -0.8\end{aligned}$$

4. Ağırlık Güncelleme

$$\begin{aligned}w_{new} &= w - \eta \frac{\partial L}{\partial w} \\ &= 0.3 - 0.1(-0.8) = 0.38\end{aligned}$$

Optimizasyon (Gradyan İnişi Algoritması)

- Optimizasyon hesaplanan kayıp değerini (hatayı) en aza indirmek için ağdaki çok sayıda ağırlık ve bias değerinin matematikle güncellenme sürecidir.
- **Benzetme:** Optimizasyon gözü bağlı bir şekilde dağın zirvesinden (yüksek hata) en derin vadiye (düşük hata) doğru adım adım inmeye benzer.
- Adımların yönünü ve büyüklüğünü **Gradyan (Eğim/Türev)** belirler. Bu işleme **Gradyan İnişi (Gradient Descent)** denir.

Optimizasyon ve Öğrenme Döngüsü Nasıl Çalışır?

- **1. İleri Yayılım (Forward Pass):** Veri ağa girer ve matematiksel işlemlerden geçerek bir tahmin (`y_pred`) üretir.
- **2. Kayıp Hesabı (Loss Calculation):** Tahmin ile gerçek hedef (`y_true`) arasındaki sapma Kayıp Fonksiyonu ile hesaplanır.
- **3. Geri Yayılım (Backpropagation): Zincir Kuralı (Chain Rule)** kullanılarak hatanın sorumluluğu çıkış katmanından giriş katmanına doğru ağırlık paylaştırılır (Türevler hesaplanır).
- **4. Optimizasyon (Weight Update):** Optimizer algoritması türev bilgisini kullanarak ağırlıkları hatayı azaltacak şekilde bir miktar değiştirir.

Bu adımlar model verideki örüntüyü tam olarak öğrenene kadar defalarca kez tekrarlanır.

Optimizasyon Algoritmaları

- Sinir ağı öğrenirken ana amaç **kayıp (loss)** fonksiyonunu en aza indirmektir.
- Geri yayılım ile hesaplanan **gradyanlar (türevler)** kullanılarak ağırlıklar güncellenir.
- Güncellemeyi yapan algoritmaya **Optimizer (İyileştirici)** denir.

- **SGD** (Stochastic Gradient Descent)
- **Adam**
- **RMSProp**

Doğru optimizer hızlı ve dengeli öğrenme sağlar.

Derin Öğrenmede Optimizer Algoritmaları

1. SGD (Stochastic Gradient Descent - Stokastik Gradyan İnişi)

- Ağırlıkları her örnekte sabit bir hızla günceller. Yerel çukurlara (Local Minima) takılma riski vardır.

2. RMSprop (Root Mean Square Propagation)

- Öğrenme hızını her parametre için otomatik olarak adapte eder. Zaman Serilerinde (RNN) başarılıdır.

3. Adam (Adaptive Moment Estimation)

- Hızlı yakınsar, daha az hiperparametre ayarı gerektirir ve çoğu problemde varsayılan (default) olarak seçilir.

SGD, Adam ve RMSProp Karşılaştırması

Özellik	SGD	Adam	RMSProp
Hız	Yavaş	Hızlı	Orta
Bellek Kullanımı	Az	Orta	Orta
Uyarlamalı Adım	Hayır	Evet	Evet
Kararlılık	Düşük	Yüksek	Orta

Keras ile Modelin Derlenmesi (Compile İşlemi)

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

- **Not:** `compile` metodu modeli eğitmez. Modelin arka plandaki hesaplama grafiğini (Computational Graph) ve kuralları belirler.
- `optimizer="adam"`: Ağırlıkların nasıl güncelleneceğini belirten yöntem.
- `loss="categorical_crossentropy"`: Hatanın nasıl ölçüleceğini belirten formül.
- `metrics=["accuracy"]`: Başarı metriği.

Model Eğitimi (`fit` Süreci)

- Model `fit()` fonksiyonu aracılığıyla verideki örüntüleri öğrenmeye başlar.
- **Adımlar**
  - Girdi verisi (`X_train`) ağa beslenir.
  - Model bir tahmin yapar ve bu tahmini gerçek etiketlerle (`y_train`) kıyaslayarak **kayıp (loss)** değerini hesaplar.
  - Optimizasyon algoritması kaybı azaltacak şekilde ağdaki ağırlıkları geriye doğru günceller.
- Eğitim sırasında kayıp değeri düzenli bir şekilde azalıyorsa model doğru öğreniyor demektir.

Örnek Keras Kullanımı

```
history = model.fit(X_train, y_train, epochs=10, batch_size=32)
```

Hiperparametreler

- **Modelin parametreleri (Ağırlıklar ve Biaslar):** Veriden kendi kendine öğrenilen ve güncellenen matematiksel değerlerdir.
- **Hiperparametreler:** Modelin dışında yer alan **önceden belirlenen** ve öğrenme sürecini kontrol eden ayarlardır.
- Doğru ayarlanmış hiperparametreler modelin başarısını artırır. Yanlış ayarlar modelin çökmesine neden olur.

Temel Hiperparametreler

- **Epoch:** Tüm veri setinin modele kaç kez gösterileceği.
- **Batch Size:** Ağırlıklar güncellenmeden önce kaç adet verinin işleneceği.
- **Learning Rate:** Ağırlıkların her adımda ne kadar şiddetle güncelleneceği.

Epoch (Eğitim Tekrar Sayısı)

- **Epoch:** Tüm eğitim verisinin modelin içinden **baştan sona tam bir tur** geçmesi işlemidir.
- Bir kitabı bir kez okumak 1 Epoch demektir. Sınava hazırlanan bir öğrenci gibi model de karmaşık bir konuyu anlamak için kitabı defalarca okumak (10-50 Epoch) zorundadır.

Denge

- **Çok Az Epoch:** Model veriyi tam olarak öğrenemez, temel kuralları kaçırır (**Underfitting**).
- **Çok Fazla Epoch:** Model verinin genel kurallarını öğrenemez. Tüm soruları cevaplarıyla ezberlemeye başlar (**Overfitting**).

Batch Size (Mini Grup Eğitimi)

- Veri seti yüz binlerce resimden oluşuyorsa hepsini aynı anda RAM'e (veya GPU'ya) sığdırmak imkansızdır.
- Veriler modele **parça parça (paketler halinde)** sunulur. Bu paketlerin boyutuna **Batch Size** denir.
- Model her bir paketi inceledikten sonra ağırlıklarını günceller.

Etkisi

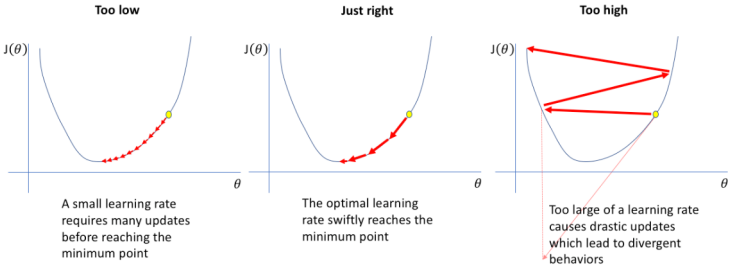
- **Küçük Batch (Örn: 8, 16):** Daha gürültülü ama çok güncelleme yapar, yerel çukurlardan kolay kurtulur.
- **Büyük Batch (Örn: 128, 256):** Daha pürüzsüz ve hızlı öğrenir ancak çok fazla donanım (RAM/VRAM) belleği gerektirir.

Learning Rate (Öğrenme Hızı)

- Modelin hatasını düzeltebilmesi için optimizasyon dağından aşağıya doğru attığı **adımların büyüklüğüdür**.

- **Çok Küçük (Örn: 0.00001):** Öğrenme işlemi saatler, günler sürebilir. Model çok yavaş ilerlediği için yerel bir çukura saplanıp kalabilir.
- **Çok Büyük (Örn: 0.1):** Model çok büyük adımlarla öğrendiği için optimum noktayı bulamayabilir.
- **İdeal Çözüm:** Büyük adımlarla başlayıp optimuma yaklaştıkça adımları küçülten **Adam** gibi adaptif optimizasyon algoritmaları kullanmaktır.

Learning Rate Etkisi



Epoch ve Batch Size

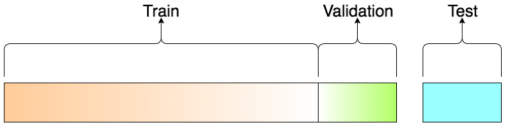
```
# Deney 1: Standart ayarlar
model.fit(X_train, y_train,
          epochs=10, batch_size=32,
          validation_split=0.2)

# Deney 2: Daha kucuk batch, daha uzun epoch
model.fit(X_train, y_train,
          epochs=50, batch_size=8,
          validation_split=0.2)
```

- Çok düşük batch size daha yavaş ama daha hassas öğrenme sağlar.
- Fazla epoch Early Stopping yoksa aşırı öğrenmeye (overfitting) neden olabilir.

Eğitim ve Doğrulama (Validation) Ayrımı

- Model **Eğitim (Train)** verisi ile öğrenir.
- Model eğitim verisiyle test edilirse cevapların **ezberleyip ezberlemediğini** ölçülmüş olur.
- Çözüm:
  - **Eğitim Seti:** Ağırlıkların güncellendiği asıl öğrenme alanıdır.
  - **Doğrulama (Validation) Seti:** Her Epoch sonunda ezberlemeyi (Overfitting) takip etmek için kullanılan gözetleme setidir.
  - **Test Seti:** Model bittikten sonra **gerçek dünya** değerlendirmesi için saklanan izole veridir.

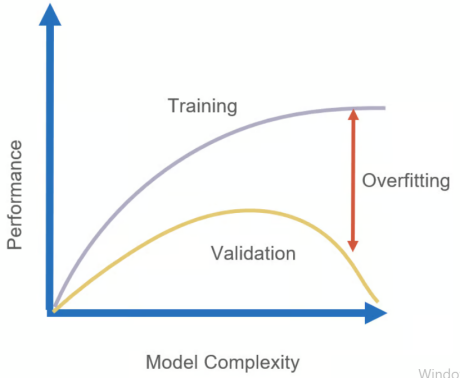
Eğitim, Doğrulama ve Test Mimarisi



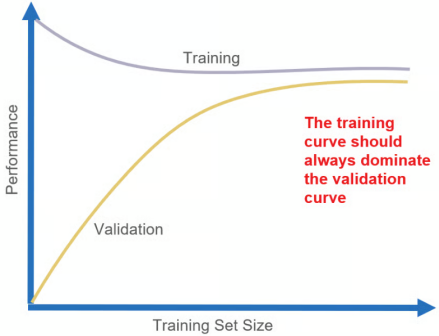
Model Eğitimi Sırasında İzleme (Öğrenme Eğrisi)

- `fit()` komutu çalıştığında arka planda eğitim verisi (`loss`) ve doğrulama verisi (`val_loss`) üzerindeki hata payları her Epoch sonunda kaydedilir.
- Bu iki çizginin aynı grafik üzerinde çizilmesine **Öğrenme Eğrisi (Learning Curve)** denir.

Aşırı Öğrenme (Overfitting) Tespiti



Optimum Nokta



Modelin Test Verisi ile Değerlendirilmesi

- Eğitimden sonra modelin başarısı daha önce hiç görmediği test verisiyle ölçülür.
- `evaluate()` fonksiyonu test seti üzerindeki doğruluk ve kayıp değerlerini verir.

Kod Örneği

```
loss, acc = model.evaluate(X_test, y_test)
print(f"Test Dogruluk: {acc:.2f}")
```

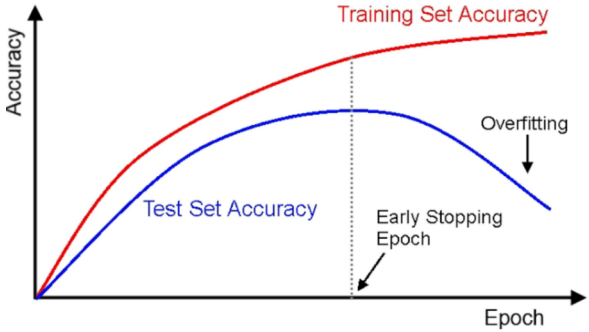
Test verisinde elde edilen başarı modelin ezberlemediğini ve genelleme gücünün yüksek olduğunu gösterir.

Erken Durdurma (Early Stopping)

- Eğitim sırasında doğrulama kaybı (validation loss) iyileşmeyi durdurur ve artmaya başlarsa:
  - Model aşırı öğrenmeye (ezberlemeye) başlar.
  - Artık genelleştirilebilir bir öğrenme gerçekleşmez.
- **Erken durdurma (Early Stopping)** bu durumda eğitimi otomatik olarak sonlandırır.
- Eğitim süresi kısalır, modelin genelleme kapasitesi korunur.

Kullanım: `EarlyStopping(monitor=val_loss, patience=3)`

Erken Durdurma



EarlyStopping Kullanımı

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor="val_loss", patience=3, restore_best_weights=True)

model.fit(X_train, y_train,
          epochs=50, batch_size=32,
          validation_split=0.2,
          callbacks=[early_stop])
```

3 epoch boyunca `val_loss` değerinde iyileşme olmazsa eğitim durdurulur ve en başarılı ağırlıklar geri yüklenir.

Loss / Accuracy Grafikleri (Öğrenme Eğrisi)

```
history = model.fit(...)

plt.plot(history.history["loss"], label="Egitim Kaybi")
plt.plot(history.history["val_loss"], label="Dogrulama Kaybi")
plt.legend()
plt.title("Kayip Grafigi")
plt.show()
```

Eğitim ve doğrulama eğrileri her zaman birlikte analiz edilmelidir. Sadece eğitim eğrisine bakmak yanıltıcıdır.

Iris Projesi: Veri Yükleme ve Ön İşleme

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

X, y = load_iris(return_X_y=True)
y = to_categorical(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Iris Projesi: Model Tanımı (Dropout ve L2)

```
from tensorflow.keras.regularizers import l2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, InputLayer

model = Sequential([
    InputLayer(input_shape=(4, )),
    Dense(64, activation="relu", kernel_regularizer=l2(0.01)),
    Dropout(0.5),
    Dense(3, activation="softmax")
])
```

Iris Projesi: Model Derleme (Compile)

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

- `adam`: Adaptif öğrenme oranı ile hızlı optimizasyon sağlar.
- `categorical_crossentropy`: Çok sınıflı (Multi-class) sınıflandırma için ideal kayıp fonksiyonudur.
- `metrics=["accuracy"]`: Başarı ölçütü olarak doğruluk kullanılır.

Iris Projesi: Model Eğitimi (Fit)

```
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=8,
    verbose=1
)
```

- Eğitim verisinin %20'si doğrulama (validation) için ayrılır.
- Model 50 epoch boyunca küçük gruplar (batch_size=8) halinde öğrenir.

Sinir Ağı Eğitimi (Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
  Dense(16, activation="relu", input_shape=(X_train.shape[1],)),
  Dense(8, activation="relu"),
  Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

history = model.fit(X_train, y_train,
                    epochs=20, batch_size=32,
                    validation_split=0.2)
```

Iris Projesi: Modeli Değerlendirme ve Tahmin

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test dogrulugu: {test_acc:.2f}")

sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)
print("Tahmin Olasiliklari:", prediction)
```

- `evaluate()`: modelin test setindeki genel başarısını ölçer.
- `predict()`: yeni gelen bir örnek için sınıflara ait olasılıkları verir.

Proje: Fashion MNIST Veriseti Tanıtımı

- **Fashion MNIST** MNIST rakam veri setinin bir versiyonudur.
- Toplam **70.000 adet** gri tonlamalı **28x28 piksel** giysi görseli içerir.
- 10 farklı sınıfa ayrılmıştır (her sınıf 7.000 örnek)
  - 0 = T-shirt/top 1 = Trouser 2 = Pullover
  - 3 = Dress 4 = Coat 5 = Sandal
  - 6 = Shirt 7 = Sneaker 8 = Bag 9 = Ankle boot
- Eğitim seti: 60.000 örnek, Test seti: 10.000 örnek

1. Fashion MNIST Veri Setini Yükleme

```
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print(X_train.shape, y_train.shape)

plt.imshow(X_train[0], cmap="gray")
plt.title(f"Label: {y_train[0]}")
plt.show()
```

2. Görselleri Ön İşleme (Preprocessing)

```
# Olcekleme: Pikselleri 0-1 araligina cekmek (Max deger 255)
X_train = X_train / 255.0
X_test = X_test / 255.0

# Duzlestirme: 28x28 matrisi 784 boyutlu vektore cevirme
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# Etiketleri One-Hot formatina dondurme
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

- Görsel veriyi normalize etmek (255'e bölmek) öğrenmeyi çok hızlandırır.
- **One-hot encoding**: 10 sınıf için uygun çıkış formatıdır.

3. MLP Modeli Oluşturma

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

model = Sequential([
    Dense(256, activation="relu", input_shape=(784,)),
    Dropout(0.4),
    Dense(128, activation="relu"),
    Dropout(0.3),
    Dense(10, activation="softmax")
])
```

- **Giriş:** 784 nöron (düzleştirilmiş görsel).
- **Gizli Katmanlar:** 2 adet ve Dropout ile ezberlemeyi önleme mekanizması.
- **Çıkış:** 10 sınıf için Softmax aktivasyonu.

4. Modeli Derleme ve Eğitme

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train, y_train,
    epochs=15,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
```

- `validation_split=0.1`: Eğitim verisinin %10'u doğrulama (validation) için ayrılır.

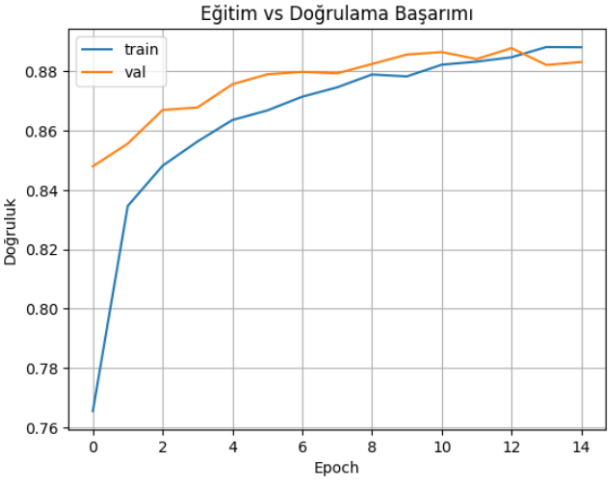
5. Test Başarımı ve Tahmin

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test dogrulugu: {test_acc:.4f}")

import numpy as np
idx = np.random.randint(0, len(X_test))
prediction = model.predict(X_test[idx].reshape(1, 784))

print("Tahmin Edilen Sinif:", prediction.argmax())
```

- Test doğruluğu genellikle Fashion MNIST için %87–89 bandında çıkar.
- `argmax()`: 10 farklı olasılık içinden en yüksek olasılığı veren sınıfın indeksini seçer.

6. Eğitim ve Doğrulama Doğruluk Grafiği

```
import matplotlib.pyplot as plt

plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="val")
plt.xlabel("Epoch")
plt.ylabel("Dogruluk")
plt.title("Egitim vs Dogrulama Basarimi")
plt.legend()
plt.grid(True)
plt.show()
```

- Modelin aşırı öğrenip öğrenmediği (overfitting) bu grafikle anlaşılır.
- Eğitimin sonunda doğrulama başarımı duruyorsa erken durdurma (early stopping) düşünülebilir.

Uygulama Sonucu ve Çıktılar